

[Euron] 머신러닝 스터디 2주차

[통계](#) [수정](#) [삭제](#)

조운서 · 방금 전

❤️ 0

교재: 파이썬 머신러닝 완벽 가이드 (권철민)

제 2장. 사이킷런으로 시작하는 머신러닝

01. 사이킷런 소개와 특징

- 사이킷런: 파이썬 머신러닝 라이브러리

- 가장 많이 사용되는 머신러닝 라이브러리
- 가장 쉽고 파이썬스러운 API 제공
- 머신러닝을 위한 매우 다양한 알고리즘과 개발을 위한 편리한 프레임워크&API를 제공

```
import sklearn
```

02. 첫 번째 머신러닝 만들어 보기 - 붓꽃 품종 예측하기

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.model_selection import train_test_split
```

- `sklearn.datasets` : 사이킷런에서 자체적으로 제공하는 데이터 세트를 생성하는 모듈의 모임
- `sklearn.tree` : 트리 기반 머신러닝 알고리즘을 구현한 클래스의 모임
- `sklearn.model_selection` : 학습 데이터와 검증 데이터, 예측 데이터로 데이터를 분리 / 최적의 하이퍼 파라미터로 평가하기 위한 다양한 모듈의 모임
- `train_test_split()` : 데이터를 학습 데이터와 테스트 데이터로 분리
- `load_iris()` : 붓꽃 데이터 세트를 생성

```
import pandas as pd

iris = load_iris()

iris_data = iris.data

iris_label = iris.target
print('iris target값:', iris_label)
print('iris target명:', iris.target_names)

iris_df = pd.DataFrame(data=iris_data, columns=iris.feature_names)
iris_df['label'] = iris.target
iris_df.head(3)
```

```
iris.target값: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2  
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
2 2]  
iris.target명: ['setosa' 'versicolor' 'virginica']
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0

- 피쳐 - sepal length, sepal width, petal length, petal width
- 레이블 - 0, 1, 2 (Setosa, versicolor, virginica)

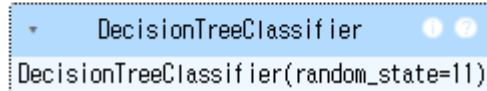
[illegible]

- `iris_data` 는 피쳐 데이터 세트
- `iris_label` 은 레이블 데이터 세트
- `test_size` : 전체 데이터 세트 중 테스트 데이터 세트의 비율
↳ 테스트 데이터 20%, 학습 데이터 80%

- `random_state` : 호출할 때마다 같은 학습/테스트 용 데이터 세트를 생성하기 위해 주어지는 난수

1) 학습 수행

```
df_clf = DecisionTreeClassifier(random_state=11)
df_clf.fit(X_train, y_train)
```



```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=11)
```

- `DecisionTreeClassifier.fit(학습용 피쳐 데이터 속성, 결정값 데이터 세트)` : 의사 결정 트리 학습

2) 예측 수행

```
pred = df_clf.predict(X_test)
```

- `predict(테스트용 피쳐 데이터 세트)` : 학습된 모델 기반으로 예측값 반환

3) 예측 성능 평가

```
from sklearn.metrics import accuracy_score
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

```
예측 정확도: 0.9333
```

- `accuracy_score(실제 레이블 데이터 세트, 예측 레이블 데이터 세트)` : 예측된 레이블 값과 실제 레이블 값 예측 정확도 평가

03. 사이킷런의 기반 프레임워크 익히기

Estimator 이해

- **Estimator**: 지도학습의 모든 알고리즘을 구현한 클래스(Classifier와 Regressor)
- 분류 - DecisionTreeClassifier, RandomForestClassifier, GradientBoostingClassifier, GaussianNB, SVC
- 회귀 - LinearRegression, Ridge, Lasso, RandomFroestRegressor, GradientBoostingRegressor

사이킷런의 주요 모듈

	모듈명	설명
예제 데이터	<code>sklearn.datasets</code>	사이킷런에 내장되어 예제로 제공하는 데이터 세트
피처 처리	<code>sklearn.preprocessing</code>	데이터 전처리에 필요한 다양한 가공 기능 제공(문자열을 숫자형 코드 값으로 인코딩, 정규화, 스케일링 등)
-	<code>sklearn.feature_selection</code>	알고리즘에 큰 영향을 미치는 피처를 우선순위로 선택 작업을 수행하는 다양한 기능 제공
-	<code>sklearn.feature_extraction</code>	텍스트 데이터/이미지 데이터의 벡터화된 피처를 추출하는데 사용됨
피처 처리 & 차원 축소	<code>sklearn.decomposition</code>	차원 축소와 관련된 알고리즘을 지원하는 모듈
데이터 분리, 검증 & 파라미터 튜닝	<code>sklearn.model_selection</code>	교차 검증을 위한 학습용/테스트용 분리
평가	<code>sklearn.metrics</code>	분류, 회귀, 클러스터링, 페어와이즈에대한 다양한 성능 측정 방법 제공
ML 알고리즘	<code>sklearn.ensemble</code>	앙상블 알고리즘 제공(랜덤 포레스트, 에이다 부스팅, 그래디언트 부스팅 등)
-	<code>sklearn.linear_model</code>	주로 선형 회귀, 릿지, 라쏘 및 로지스틱 회귀 등 회귀 관련 알고리즘을 지원 + SGD 관련 알고리즘 제공

	모듈명	설명
-	<code>sklearn.naive_bayes</code>	나이브 베이즈 알고리즘 제공(가우시안 NB, 다항 분포 NB 등)
-	<code>sklearn.neighbors</code>	최근접 이웃 알고리즘 제공(K-NN 등)
-	<code>sklearn.svm</code>	서포트 벡터 머신 알고리즘 제공
-	<code>sklearn.tree</code>	의사 결정 트리 알고리즘 제공
-	<code>sklearn.cluster</code>	비지도 클러스터링 알고리즘 제공 (K-means, 계층형, DBSCAN 등)
유틸리티	<code>sklearn.pipeline</code>	피처 처리 등의 변환과 ML 알고리즘 학습, 예측 등을 함께 묶어서 실행할 수 있는 유틸리티 제공

내장된 예제 데이터 세트

API 명	용도	설명
<code>datasets.load_boston()</code>	회귀	미국 보스턴의 집 피쳐 & 가격에 대한 데이터 세트
<code>datasets.load_breast_cancer()</code>	분류	위스콘신 유방암 피쳐 & 악성/양성 레이블 데이터 세트
<code>datasets.load_diabetes()</code>	회귀	당뇨 데이터 세트
<code>datasets.load_digits()</code>	분류	0에서 9까지 숫자의 이미지 픽셀 데이터 세트
<code>datasets.load_iris()</code>	분류	붓꽃에 대한 피쳐를 가진 데이터 세트

API 명	목적
<code>datasets.make_classification()</code>	분류
<code>datasets.make_blobs()</code>	클러스터링

- data : 피처의 데이터 세트 (ndarray)
- target : 분류 시 레이블 값, 회귀일 때는 숫자 결괏값 데이터 세트 (ndarray)
- target_names : 개별 레이블의 이름 (ndarray 또는 list)
- feature_names : 피처의 이름 (ndarray 또는 list)
- DESCR : 데이터 세트에 대한 설명 & 각 피처의 설명 (string)

04. Model Selection 모듈 소개

- **model_selection** : 학습 데이터와 테스트 데이터 세트를 분리하거나 교차 검증 분할 및 평가, Estimator의 하이퍼 파라미터를 튜닝하기 위한 다양한 함수와 클래스를 제공하는 모듈

학습/테스트 데이터 세트 분리

- **train_test_split()** : 학습 데이터와 테스트 데이터 세트를 분리

데이터 세트를 분리하지 않았을 때

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
dt_clf = DecisionTreeClassifier()
train_data = iris.data
train_label = iris.target
dt_clf.fit(train_data, train_label)

pred = dt_clf.predict(train_data)
print('예측 정확도:', accuracy_score(train_label, pred))
```

예측 정확도: 1.0

- 예측 정확도가 100% -> 이상함

**** 데이터 세트를 분리**

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

dt_clf = DecisionTreeClassifier()
iris_data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size=0.3, random_state=121)

dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))

예측 정확도: 0.9556

```

- 예측 정확도가 약 95.56%

교차 검증

- 고정된 학습 데이터와 테스트 데이터로 평가 -> 테스트 데이터에만 최적의 성능을 발휘하도록 평향되게 모델을 유도하는 경향이 생김 -> **과적합** -> 다른 테스트용 데이터가 들어올 경우, 성능 ↓
- **교차 검증**: 데이터 편종을 막기 위해 별도의 여러 세트로 구성된 학습 데이터 세트와 검증 데이터 세트에서 학습과 평가를 수행
- **K 폴드 교차 검증**



- K개의 데이터 폴드 세트를 만들어 K번만큼 각 폴드 세트에 학습과 검증 평가를 반복적으로 수행

- `KFold(n_splits=k)` : k개의 폴드 세트로 분리하는 객체 생성

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import KFold
import numpy as np

iris = load_iris()
features = iris.data
label = iris.target
dt_clf = DecisionTreeClassifier(random_state=156)

kfold = KFold(n_splits=5)
cv_accuracy = []
print('붓꽃 데이터 세트 크기:', features.shape[0])
```

붓꽃 데이터 세트 크기: 150

```

n_iter = 0

for train_index, test_index in kfold.split(features):
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]

    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)
    n_iter += 1

    accuracy = np.round(accuracy_score(y_test, pred), 4)
    train_size = X_train.shape[0]
    test_size = X_test.shape[0]
    print('\n#{0} 교차 검증 정확도: {1}, 학습 데이터 크기: {2}, 검증 데이터 크기: {3}'
          .format(n_iter, accuracy, train_size, test_size))
    print('#{0} 검증 세트 인덱스: {1}'.format(n_iter, test_index))
    cv_accuracy.append(accuracy)

print('\n## 평균 검증 정확도:', np.mean(cv_accuracy))

```

```

#1 교차 검증 정확도: 1.0, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#1 검증 세트 인덱스: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29]

#2 교차 검증 정확도: 0.9667, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#2 검증 세트 인덱스: [30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
54 55 56 57 58 59]

#3 교차 검증 정확도: 0.8667, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#3 검증 세트 인덱스: [60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83
84 85 86 87 88 89]

#4 교차 검증 정확도: 0.9333, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#4 검증 세트 인덱스: [ 90  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107
108 109 110 111 112 113 114 115 116 117 118 119]

#5 교차 검증 정확도: 0.7333, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#5 검증 세트 인덱스: [120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137
138 139 140 141 142 143 144 145 146 147 148 149]

## 평균 검증 정확도: 0.9

```

- 교차 검증 시마다 검증 세트의 인덱스가 달라짐

- Stratified K 폴드

- 불균형한 분포도를 가진 레이블 데이터 집합을 위한 K 폴드 방식
- 레이블 데이터 집합이 원본 데이터 집합의 레이블 분포를 학습 및 테스트 세트에 제대로 분배하지 못하는 경우의 문제 해결

<불균형한 분포도를 가진 레이블 데이터 문제점>

```

kfold = KFold(n_splits=3)
n_iter=0
for train_index, test_index in kfold.split(iris_df):
    n_iter += 1
    label_train=iris_df['label'].iloc[train_index]
    label_test=iris_df['label'].iloc[test_index]
    print('## 교차 검증: {0}'.format(n_iter))
    print('학습 레이블 데이터 분포:\n', label_train.value_counts())
    print('검증 레이블 데이터 분포:\n', label_test.value_counts())

```

```

## 교차 검증: 1
학습 레이블 데이터 분포:
label
1    50
2    50
Name: count, dtype: int64
검증 레이블 데이터 분포:
label
0    50
Name: count, dtype: int64
## 교차 검증: 2
학습 레이블 데이터 분포:
label
0    50
2    50
Name: count, dtype: int64
검증 레이블 데이터 분포:
label
1    50
Name: count, dtype: int64
## 교차 검증: 3
학습 레이블 데이터 분포:
label
0    50
1    50
Name: count, dtype: int64
검증 레이블 데이터 분포:
label
2    50
Name: count, dtype: int64

```

- 학습 레이블 값 ≠ 검증 레이블 값
- 학습 레이블이 1, 2 밖에 없기 때문에 학습 모델은 절대 0을 예측하지 못함 -> 검정 레이블은 0 값밖에 없음 -> 검증 예측 정확도 '0'

<해결 방안>

```
for train_index, test_index in skf.split(iris_df, iris_df['label']):
    n_iter += 1
    label_train = iris_df['label'].iloc[train_index]
    label_test = iris_df['label'].iloc[test_index]
    print('## 교차 검증: {}'.format(n_iter))
    print('학습 레이블 데이터 분포:\n', label_train.value_counts())
    print('검증 레이블 데이터 분포:\n', label_test.value_counts())
```

```
## 교차 검증: 1
학습 레이블 데이터 분포:
label
2    34
0    33
1    33
Name: count, dtype: int64
검증 레이블 데이터 분포:
label
0    17
1    17
2    16
Name: count, dtype: int64
## 교차 검증: 2
학습 레이블 데이터 분포:
label
1    34
0    33
2    33
Name: count, dtype: int64
검증 레이블 데이터 분포:
label
0    17
2    17
1    16
Name: count, dtype: int64
## 교차 검증: 3
학습 레이블 데이터 분포:
label
0    34
1    33
2    33
Name: count, dtype: int64
검증 레이블 데이터 분포:
label
1    17
2    17
0    16
Name: count, dtype: int64
```

- 학습 레이블 데이터 값의 분포도 ≈ 검증 레이블 데이터 값의 분포도
- 레이블 값 0, 1, 2를 모두 학습할 수 있고 이에 기반해 검증을 수행할 수 있음
- ∴ StratifiedKFold는 KFold로 분할된 레이블 데이터 세트가 전체 레이블 값의 분포도를 반영하지 못하는 문제를 해결

교차 검증을 보다 간편하게

- `cross_val_score()`: 교차 검증을 좀 더 편리하게 수행할 수 있게 해주는 API (하나의 평가 지표)

```
cross_val_score(estimator, X, y=None, scoring=None, cv=None,
n_jobs=1, verbose=0, fit_params=None, pre_dispatch='2*n_jobs')
```

- estimator : Classifier 또는 Regressor
- X : 피쳐 데이터 세트
- y : 레이블 데이터 세트
- scoring = 예측 성능 평가 지표
- cv : 교차 검증 폴드 수
- 수행 후 scoring 파라미터로 지정된 성능 지표 측정값을 배열 형태로 반환

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, cross_validate
from sklearn.datasets import load_iris

iris_data = load_iris()
dt_clf = DecisionTreeClassifier(random_state=156)

data = iris_data.data
label = iris_data.target

scores = cross_val_score(dt_clf, data, label, scoring='accuracy', cv=3)
print('교차 검증별 정확도:', np.round(scores, 4))
print('평균 검증 정확도:', np.round(np.mean(scores), 4))
```

```
교차 검증별 정확도: [0.98 0.94 0.98]
평균 검증 정확도: 0.9667
```

- 교차 검증 폴드 수 = 3, 성능 평가 지표 = accuracy로 지정

- `cross_validate()` : 여러 개의 평가 지표를 반환

GridSearchCV - 교차 검증과 최적 하이퍼 파라미터 튜닝을 한 번에

- **GridSearchCV API**: 편리하게 최적의 파라미터를 도출

- 사용자가 튜닝하고자 하는 여러 종류의 하이퍼 파라미터를 다양하게 테스트하면서 최적의 파라미터를 찾게 해줌
- 동시에 순차적으로 파라미터를 테스트 -> 수행시간 ↑
- GridSearchCV 클래스의 생성자로 들어가는 주요 파라미터:

파라미터	설명
estimator	classifier, regressor, pipeline이 사용될 수 있음
param_grid	key + 리스트 값을 가지는 딕셔너리가 주어짐, estimator의 튜닝을 위해 파라미터명과 사용될 여러 파라미터 값을 지정
scoring	예측 성능을 측정할 평가 방법을 지정
cv	교차 검증을 위해 분할되는 학습/테스트 세트 개수를 지정
refit	디폴트가 True, True로 생성 시 가장 최적의 하이퍼 파라미터를 찾은 뒤 입력된 estimator 객체를 해당 하이퍼파라미터로 재학습

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

iris_data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size=0.2, random_state=121)

dtree = DecisionTreeClassifier()

parameters = {'max_depth': [1, 2, 3], 'min_samples_split': [2, 3]}

```

```

import pandas as pd

grid_dtree = GridSearchCV(dtree, param_grid=parameters, cv=3, refit=True)

grid_dtree.fit(X_train, y_train)

scores_df = pd.DataFrame(grid_dtree.cv_results_)
scores_df[['params', 'mean_test_score', 'rank_test_score',
            'split0_test_score', 'split1_test_score', 'split2_test_score']]

```

	params	mean_test_score	rank_test_score	split0_test_score	split1_test_score	split2_test_score
0	{'max_depth': 1, 'min_samples_split': 2}	0.700000	5	0.700	0.7	0.70
1	{'max_depth': 1, 'min_samples_split': 3}	0.700000	5	0.700	0.7	0.70
2	{'max_depth': 2, 'min_samples_split': 2}	0.958333	3	0.925	1.0	0.95
3	{'max_depth': 2, 'min_samples_split': 3}	0.958333	3	0.925	1.0	0.95
4	{'max_depth': 3, 'min_samples_split': 2}	0.975000	1	0.975	1.0	0.95
5	{'max_depth': 3, 'min_samples_split': 3}	0.975000	1	0.975	1.0	0.95

- 하이퍼 파라미터 max_depth와 min_samples_split을 순차적으로 총 6번 변경하면서 학습 및 평가 수행

```
print('GridSearchCV 최적 파라미터:', grid_dtrees.best_params_)
print('GridSearchCV 최고 정확도: {0:.4f}'.format(grid_dtrees.best_score_))

GridSearchCV 최적 파라미터: {'max_depth': 3, 'min_samples_split': 2}
GridSearchCV 최고 정확도: 0.9750
```

- max_depth가 3, min_samples_split이 2일 때 검증용 폴드 세트에서 평균 최고 정확도가 97,50%로 측정됨

05. 데이터 전처리

- 결손값 NaN, Null 값은 허용X -> 고정된 다른 값으로 변환
- 사이킷런의 머신러닝 알고리즘은 문자열 값을 입력값으로 허용X -> 모든 문자열 값은 인코딩해서 숫자 형으로 변환

데이터 인코딩

레이블 인코딩: 카테고리 피처를 코드형 숫자 값으로 변환

- 숫자 값의 경우 크고 작음에 대한 특성이 작용 -> 선형회귀와 같은 ML 알고리즘에는 적용하면 예측 성능 ↓

```
from sklearn.preprocessing import LabelEncoder

items = ['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '선풍기', '믹서', '믹서']

encoder = LabelEncoder()
encoder.fit(items)
labels = encoder.transform(items)
print('인코딩 변환값:', labels)

인코딩 변환값: [0 1 4 5 3 3 2 2]

print('인코딩 클래스:', encoder.classes_)

인코딩 클래스: ['TV' '냉장고' '믹서' '선풍기' '전자레인지' '컴퓨터']
```

원-핫 인코딩: 피쳐 값의 유형에 따라 새로운 피쳐를 추가해 고유 값에 해당하는 칼럼에만 1을 표시하고 나머지 칼럼에는 0을 표시하는 방식

- 행 형태로 되어 있는 피쳐의 고유 값을 열 형태로 차원을 변환

![]

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

items = ['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '선풍기', '믹서', '믹서']

items = np.array(items).reshape(-1, 1)

oh_encoder = OneHotEncoder()
oh_encoder.fit(items)
oh_labels = oh_encoder.transform(items)

print('원-핫 인코딩 데이터')
print(oh_labels.toarray())
print('원-핫 인코딩 데이터 차원')
print(oh_labels.shape)
```

```
원-핫 인코딩 데이터
[[1.  0.  0.  0.  0.  0.]
 [0.  1.  0.  0.  0.  0.]
 [0.  0.  0.  0.  1.  0.]
 [0.  0.  0.  0.  0.  1.]
 [0.  0.  0.  1.  0.  0.]
 [0.  0.  0.  1.  0.  0.]
 [0.  0.  1.  0.  0.  0.]
 [0.  0.  1.  0.  0.  0.]]
원-핫 인코딩 데이터 차원
(8, 6)
```

```
from scipy.sparse.csgraph import depth_first_tree
import pandas as pd

df = pd.DataFrame({'item':['TV', '냉장고', '전자레인지', '컴퓨터',
                           '선풍기', '선풍기', '믹서', '믹서']})
pd.get_dummies(df)
```

	item_TV	item_냉장고	item_믹서	item_선풍기	item_전자레인지	item_컴퓨터
0	True	False	False	False	False	False
1	False	True	False	False	False	False
2	False	False	False	False	True	False
3	False	False	False	False	False	True
4	False	False	False	True	False	False
5	False	False	False	True	False	False
6	False	False	True	False	False	False
7	False	False	True	False	False	False



- `get_dummies()` : 문자열 카테고리 값을 숫자 형으로 변환할 필요 없이 바로 변환

피처 스케일링과 정규화

- 피처 스케일링: 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업
- 표준화: 데이터 피처 각각이 평균이 0, 분산이 1인 가우시안 정규 분포를 가진 값으로 변환

$$x_{i_new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

- 정규화: 서로 다른 피처의 크기를 통일하기 위해 크기를 변환

$$x_{i_new} = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

StandardScaler

- 표준화를 쉽게 지원하기 위한 클래스
- 서포트 벡터 머신이나 선형 회귀, 로지스틱 회귀는 사전에 표준화를 적용하는 것이 중요

```
print('feature 들의 평균 값')
print(iris_df.mean())
print('feature 들의 분산 값')
print(iris_df.var(0))
```

```
feature 들의 평균 값
sepal length (cm)    5.843333
sepal width (cm)     3.057333
petal length (cm)    3.758000
petal width (cm)     1.199333
dtype: float64
```

```
feature 들의 분산 값
sepal length (cm)    0.685694
sepal width (cm)     0.189979
petal length (cm)    3.116278
petal width (cm)     0.581006
dtype: float64
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)
```

```
iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print('feature 들의 평균 값')
print(iris_df_scaled.mean())
print('feature 들의 분산 값')
print(iris_df_scaled.var())
```

```
feature 들의 평균 값
sepal length (cm)    -1.690315e-15
sepal width (cm)     -1.842970e-15
petal length (cm)    -1.698641e-15
petal width (cm)     -1.409243e-15
dtype: float64
```

```
feature 들의 분산 값
sepal length (cm)    1.006711
sepal width (cm)     1.006711
petal length (cm)    1.006711
petal width (cm)     1.006711
dtype: float64
```

- 모든 칼럼 값의 평균이 0에 아주 가까운 값으로, 분산은 1에 아주 가까운 값으로 변환됨

MinMaxScaler

- 데이터 값을 0과 1사이의 범위 값으로 변환(음수 값이 있으면 -1에서 1값으로 변환)
- 데이터 분포가 가우시안 분포가 아닐 경우 적용

```

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print('feature들의 최솟값')
print(iris_df_scaled.min())
print('feature들의 최댓값')
print(iris_df_scaled.max())

```

```

feature들의 최솟값
sepal length (cm)    0.0
sepal width (cm)     0.0
petal length (cm)    0.0
petal width (cm)     0.0
dtype: float64

feature들의 최댓값
sepal length (cm)    1.0
sepal width (cm)     1.0
petal length (cm)    1.0
petal width (cm)     1.0
dtype: float64

```

- 모든 피처가 0에서 1 사이의 값으로 변환됨

※ `fit()`, `transform()`, `fit_transform()` 을 이용해 스케일링 변환 시 유의할 점

- 가능하다면 전체 데이터의 스케일링 변환을 적용한 뒤 학습과 테스트 데이터로 분리
- 1이 여의치 않다면 테스트 변환 시에는 `fit()` 이나, `fit_transform()` 을 적용하지 않고 학습 데이터로 이미 `fit()` 이나 된 Scaler 객체를 이용해, `transform()` 으로 변환

제 3장. 평가

- 이진 분류: 긍정/부정과 같은 2개의 결과값만을 가짐
- 멀티 분류: 여러 개의 결정 클래스 값을 가짐

- 정확도
- 오차행렬
- 정밀도
- 재현율

- F1 스코어
- ROC AUC

01. 정확도(Accuracy

- **정확도**: 실제 데이터에서 예측 데이터가 얼마나 같은지를 판단하는 지표

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

```
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, fakepred)

array([[405,  0],
       [ 45,  0]])
```

- 불균형한 레이블 값 분포에서 ML 모델의 성능을 판단할 경우 적합한 평가 지표X

02. 오차 행렬

- **오차 행렬**: 학습된 분류 모델이 예측을 수행하면서 얼마나 헛갈리고 있는지도 함께 보여주는 지표

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

- 4분면 행렬에서 실제 레이블 클래스 값과 예측 레이블 클래스 값이 어떠한 유형을 가지고 매핑되는지를 나타냄

```
from sklearn.metrics import f1_score
f1 = f1_score(y_test, pred)
print('F1 스코어: {:.4f}'.format(f1))
```

F1 스코어: 0.9348

- ndarray 형태로 출력
 - TN은 전체 450건 데이터 중 무조건 Negative 0으로 예측해서 True가 된 결과 405건, FP는 Positive 1로 예측한 건수가 없으므로 0건
 - FN은 Positive 1인 건수 45건을 Negative로 예측해서 False가 된 결과 45건, TP는 Positive 1로 예측한 건수가 없으므로 0건
- TP, TN, FP, FN 값을 조합해 정확도, 정밀도, 재현율 값을 알 수 있음

03. 정밀도와 재현율

Positive 데이터 세트의 예측 성능에 좀 더 초점을 맞춘 평가 지표

정밀도(=양성 예측도)	재현율(민감도, TPR)
TP / (FP + TP)	TP / (FN + TP)

정밀도(=양성 예측도)	재현율(민감도, TPR)
실제 Negative인 데이터를 Positive로 잘못 판단하게 되면 업무상 큰 영향이 발생	제 Positive 데이터를 Negative로 잘못 판단하게 되면 업무상 큰 영향이 발생
TP를 높이고 FP를 낮추는 데 초점	TP를 높이고 FN을 낮추는 데 초점

정밀도/재현율 트레이드 오프

정밀도와 재현율은 상호 보완적인 평가 지표

- `predict_proba()` : 테스트 피쳐 데이터 세트를 파라미터로 입력해주면 테스트 피쳐 레코드의 개별 클래스 예측 확률 반환

04. F1 스코어

F1 스코어는 정밀도와 재현율을 결합한 지표

- 정밀도와 재현율이 어느 한 쪽으로 치우치지 않는 수치를 나타낼 때 상대적으로 높은 값을 가짐

```
from sklearn.metrics import roc_auc_score

pred_proba = lr_clf.predict_proba(X_test)[: , 1]
roc_score = roc_auc_score(y_test, pred_proba)
print('ROC AUC 값: {:.4f}'.format(roc_score))
```

ROC AUC 값: 0.9976

 업로드 중..

05. ROC 곡선과 AUC

- **ROC 곡선(= 판단 곡선)**: FPR이 변할 때 TPR이 어떻게 변하는지를 나타내는 곡선
 - ROC 곡선이 가운데 직선에서 멀어질수록 성능 ↑
 - `roc_curve()` : ROC 곡선을 구함
- **AUC**: ROC 곡선 밑의 면적을 구한 것
 - 일반적으로 1에 가까울수록 좋은 수치
 - `roc_auc_score()` : AUC 값을 구함

업로드중..



조윤서



이전 포스트

[Euron] 머신러닝 스터디 1주차

0개의 댓글

댓글을 작성하세요

댓글 작성

관심 있을 만한 포스트

